

Hamiltonians created via addition produce incorrect results with `ExpvalC

<https://github.com/PennyLaneAI/pennylane/issues/1389>

ROW 19

Hamiltonians created via addition produce incorrect results with `ExpvalCost(optimize=True)` #1389

Closed

#1405



KetpuntoG opened on Jun 7, 2021

Good afternoon,

I was having problem with `vqe` when I set the flag: `optimize = True`. I noticed that there were errors when entering Hamiltonians of the form `Z1@Z2 + Z2@Z1`. We know it is the same as `2.Z1.Z2` but the optimizer seems to have some kind of problem with this kind of Hamiltonians (`vqe` with `optimize = False` works correctly in this situation).

Although by preprocessing the Hamiltonian we avoid the problem, I was writing to take the problem into account.

Greetings!



josh146 on Jun 7, 2021

Member

Thanks for reporting this [@KetpuntoG](#)! Someone from the team will help investigate and pin down this bug shortly 😊



josh146 added `bug` on Jun 7, 2021

josh146 changed the title `vqe(optimize=True)` Hamiltonians created via addition produce incorrect results with `ExpvalCost(optimize=True)` on Jun 7, 2021



antalszava on Jun 8, 2021

Contributor

Hi [@KetpuntoG](#),

Thank you for drawing our attention to this! 😊 Could you perhaps post a non-working example?

I attempted to reproduce the error, though, with a naive example, the `optimize=False` and the `optimize=True` cases seem to produce the same results:

```
import pennylane as qml
import numpy as np

dev = qml.device('default.qubit', wires=4)

coeffs = [1]
obs1 = [
    qml.PauliZ(0) @ qml.PauliZ(1)
]
H1 = qml.vqe.Hamiltonian(coeffs, obs1)

obs2 = [
    qml.PauliZ(1) @ qml.PauliZ(0)
]
H2 = qml.vqe.Hamiltonian(coeffs, obs2)

ansatz = qml.templates.StronglyEntanglingLayers
dev = qml.device("default.qubit", wires=4)

params = np.ones((3,4,3))

cost1 = qml.ExpvalCost(ansatz, H1 + H2, dev, parallel=False)
cost2 = qml.ExpvalCost(ansatz, H1 + H2, dev, parallel=True)
print(np.allclose(cost1(params), cost2(params)))
```

True



When checking the terms of the Hamiltonian that is the sum of `H1` and `H2`, it seems that we get the result you suggested we would expect:

```
print((H1 + H2).terms)
```



```
[[2], [PauliZ(wires=[0]) @ PauliZ(wires=[1])]]
```



Just for internally keeping track of bugs, removing the bug label for now, but I'll read once we've tracked down the underlying specific issue. 😊



 **antalszava** removed **bug** 🐛 on Jun 8, 2021



KetpuntoG on Jun 8, 2021 · edited by KetpuntoG

Edits ▾ Contributor Author ...

Hi! Here is part of the code. In the first part I am creating a 19 qubit Hamiltonian with random coefficients:

```
import pennylane as qml
from pennylane import numpy as np
from time import time

prob = 0.5
size = 19
shots = 1000
obs = []
for j in range(size):
    if np.random.rand() < prob : obs.append(qml.PauliZ(wires = j))

    for i in range(size):
        if np.random.rand() < prob : obs.append(qml.PauliZ(wires = j) @ qml.PauliZ(wires = i))

coefs = (np.random.rand(len(obs))-0.5)*2

H = qml.Hamiltonian(coefs, obs)

def ansatz(w, wires = list(range(size))):
    for i in wires:
        qml.RX(w[i], wires = i)

def vqe(w):
    dev2 = qml.device("default.qubit", size, shots = shots)
    cost_fn = qml.ExpvalCost(ansatz, H, dev2, optimize = False)
    return cost_fn(w)

def vqe_compact(w):
    dev2 = qml.device("default.qubit", size, shots = shots)
    cost_fn = qml.ExpvalCost(ansatz, H, dev2, optimize = True)
    return cost_fn(w)

def duration(f, w):
    time1 = time()
    print("value:", f(w))
    sol = time() - time1
    print("time:", sol)

w = np.random.rand(size)*2*np.pi

print("vqe")
duration(vqe, w)
print("vqe compact")
duration(vqe_compact, w)
```



```
print("vqe compact")
duration(vqe_compact, w)
```

In my case the output is:

```
vqe
value: -4.667471020167216
time: 50.281818866729736
vqe compact
value: -8.409149436899536
time: 3.5194497108459473
```



antalszava on Jun 9, 2021

Contributor ...

Hi @KetpuntoG, thank you! Definitely something odd going on, we'll be investigating this. Thanks again for catching and reporting it. :)



antalszava added **bug** on Jun 9, 2021



antalszava added this to the **v0.16.0** milestone on Jun 9, 2021



antalszava mentioned this in 2 pull requests on Jun 11, 2021

🔗 [Simplify Hamiltonian at the beginning of ExpvalCost #1404](#)

🔗 [Pop observables and coeffs that have been grouped already #1405](#)



antalszava on Jun 11, 2021

Contributor ...

Hi @KetpuntoG, thanks again for catching this! We now have a PR addressing the issue.

There are two relevant points that are unrelated to the bug:

- The Hamiltonian in the example has terms that appear multiple times with different coefficients. Before passing a Hamiltonian to `ExpvalCost`, it is suggested to call its `simplify` method, which will consider these equal terms and add them up.
- The `@` operator is not supported between observables that target the same wire (see [Wrong simplification of Hamiltonians #1107](#) and linked issues in that thread), therefore `qml.PauliZ(0) @ qml.PauliZ(0)` leads to undefined behaviour.

Would suggest modifying the second `for` loop by adding an extra condition (`and i!=j`):

```
for i in range(size):
    if np.random.rand() < prob and i!=j:
        obs.append(qml.PauliZ(wires = j) @ qml.PauliZ(wires = i))
```



antalszava closed this as **completed** in [#1405](#) on Jun 11, 2021